



Projects

- *By Aditya Varma*

Index

Program Question	
Project 1:	Movie Recommendation System
Project 2:	Sanke and Ladder Game
Project 3:	
Project 4:	
Project 5:	
Project 6:	

Movie Recommendation System in C

Problem Statement:

The objective of this project is to design and implement a **Movie Recommendation System** using the C programming language. The system helps users discover movies and web series from a collection of **Bollywood and Hollywood content**. Users can browse all available content, search by title or genre, view top-rated movies, sort content based on ratings, and receive personalized recommendations.

This project demonstrates the use of **arrays, structures, strings, functions, loops, conditionals, searching techniques, sorting algorithms, and menu-driven programming**.

Concepts Used:

- **Structures:** To store movie, series information such as title, genre, language, type, and rating.
- **Arrays of Structures:** To maintain a collection of movies and web series.
- **Functions:** To modularize different operations like searching, sorting, displaying, and recommending content.
- **Loops:** To traverse and process the movie database.
- **Conditional Statements (if-else):** To filter content and implement recommendation logic.
- **Switch Case:** To handle menu-driven user choices.
- **Strings (string.h):** To compare and manage movie titles, genres, and languages.
- **Searching:** To locate content by title or genre.
- **Sorting:** To arrange content based on ratings.
- **Formatted Input/Output:** To display content in a clean tabular format.

Short Description of Working:

1. The program starts by loading a predefined database of movies and web series.
2. A menu is displayed to the user with multiple options.
3. The user can:
 - View all available content.
 - Search content by title.
 - Search content by genre.
 - View top-rated movies and series.
 - Get personalized recommendations.
 - Sort content by ratings.
 - Update ratings for movies or series.
4. The search feature scans the database and displays matching content.
5. The recommendation feature suggests content based on language and rating.

6. The sorting feature arranges content from highest rated to lowest rated.
7. The rating feature allows users to update ratings of existing content.
8. The program continues to run until the user selects the Exit option.

Features:

Content Library

- Bollywood Movies
- Bollywood Series
- Hollywood Movies
- Hollywood Series

Search Features

- Search by Title
- Search by Genre

Rating Features

- Show Top Rated Content
- Update Ratings
- Sort Content by Rating

Recommendation Features

- Recommend Hindi Content
- Recommend English Content
- Suggest Highly Rated Movies and Series

Utility Features

- View Complete Database
- Menu Driven Interface
- Easy Navigation

movieRecommendationSystem.c:

```

1  /*-----
2      Project Title : Movie Recommendation System
3      Description : A console based movie and series recommendation system
4                    that allows users to:
5          1. View all content
6          2. Search by title
7          3. Search by genre
8          4. Show top rated content
9          5. Recommend content
10         6. Sort by rating
11         7. Exit
12      Concepts Used : Structures, Arrays, Strings, Functions,
13                      Loops, Conditionals, Searching, Sorting
14  -----*/
15
16  #include <stdio.h>
17  #include <string.h>
18
19  struct Movie {
20      int id;
21      char title[50];
22      char genre[20];
23      char language[20];
24      char type[20];
25      float rating;
26  };
27
28  struct Movie m[] = {
29      {101, "3 Idiots", "Comedy", "Hindi", "Movie", 8.4},
30      {102, "Dangal", "Drama", "Hindi", "Movie", 8.5},
31      {103, "Drishyam", "Thriller", "Hindi", "Movie", 8.3},
32      {104, "Chhichhore", "Comedy", "Hindi", "Movie", 8.2},
33      {201, "Panchayat", "Drama", "Hindi", "Series", 9.0},
34      {202, "Mirzapur", "Crime", "Hindi", "Series", 8.6},
35      {203, "Aspirants", "Drama", "Hindi", "Series", 9.1},
36      {301, "Interstellar", "SciFi", "English", "Movie", 8.7},
37      {302, "Inception", "SciFi", "English", "Movie", 8.8},
38      {303, "Dark Knight", "Action", "English", "Movie", 9.0},
39      {401, "Breaking Bad", "Crime", "English", "Series", 9.5},
40      {402, "Dark", "SciFi", "English", "Series", 8.8}
41  };
42
43  int n = sizeof(m) / sizeof(m[0]);
44
45  /*-----
46      Function : displayAll
47      Purpose  : Display all movies and series
48  -----*/
49  void displayAll() {
50      int i;
51
52      printf("\n-----\n");
53      printf("ID\tTITLE\t\tLANGUAGE\tTYPE\tRATING\n");
54      printf("-----\n");
55
56      for(i = 0; i < n; i++) {
57          printf("%d\t%-15s\t%-10s\t%-8s %.1f\n", m[i].id, m[i].title,
58              m[i].language, m[i].type, m[i].rating);
59      }
60  }
61

```

```

62  /*-----
63      Function : searchTitle
64      Purpose  : Search content by title
65  -----*/
66  void searchTitle() {
67      char name[50];
68      int i, found = 0;
69
70      printf("Enter Title : ");
71      scanf("%s", name);
72
73      for(i = 0; i < n; i++) {
74          if(strcmp(name, m[i].title) == 0) {
75              printf("\nFound!\n");
76              printf("Title   : %s\n", m[i].title);
77              printf("Genre   : %s\n", m[i].genre);
78              printf("Language : %s\n", m[i].language);
79              printf("Type    : %s\n", m[i].type);
80              printf("Rating  : %.1f\n", m[i].rating);
81
82              found = 1;
83              break;
84          }
85      }
86
87      if(found == 0)
88          printf("Content Not Found!\n");
89  }
90
91  /*-----
92      Function : searchGenre
93      Purpose  : Search by genre
94  -----*/
95  void searchGenre() {
96      char genre[20];
97      int i;
98
99      printf("Enter Genre : ");
100     scanf("%s", genre);
101
102     printf("\nRecommendations:\n");
103
104     for(i = 0; i < n; i++) {
105         if(strcmp(genre, m[i].genre) == 0) {
106             printf("- %s\n", m[i].title);
107         }
108     }
109 }
110
111 /*-----
112     Function : topRated
113     Purpose  : Show top rated content
114 -----*/
115 void topRated() {
116     int i;
117
118     printf("\nTop Rated Content\n");
119
120     for(i = 0; i < n; i++) {
121         if(m[i].rating >= 8.8) {
122             printf("%s (%.1f)\n", m[i].title, m[i].rating);
123         }
124     }
125 }
126

```

```

127  /*-----
128      Function : recommendMovie
129      Purpose  : Recommend based on language
130  -----*/
131  void recommendMovie() {
132      int choice,i;
133
134      printf("\n1. Hindi\n");
135      printf("2. English\n");
136
137      printf("Choose Language : ");
138      scanf("%d", &choice);
139
140      printf("\nRecommended For You\n");
141
142      for(i = 0; i < n; i++) {
143          if(choice == 1 && strcmp(m[i].language, "Hindi") == 0 && m[i].rating >= 8.5) {
144              printf("%s\n",m[i].title);
145          }
146
147          if(choice==2 && strcmp(m[i].language, "English") == 0 && m[i].rating >= 8.5) {
148              printf("%s\n", m[i].title);
149          }
150      }
151  }
152
153  /*-----
154      Function : sortRating
155      Purpose  : Sort content by rating
156  -----*/
157  void sortRating() {
158      int i, j;
159      struct Movie temp;
160
161      for(i = 0; i < n-1; i++) {
162          for(j = 0; j < n-i-1; j++) {
163              if(m[j].rating < m[j+1].rating) {
164                  temp = m[j];
165                  m[j] = m[j+1];
166                  m[j+1] = temp;
167              }
168          }
169      }
170
171      printf("\nSorted By Rating (High to Low)\n");
172
173      for(i = 0; i < n; i++) {
174          printf("%s (%.1f)\n", m[i].title, m[i].rating);
175      }
176  }
177
178  /*-----
179      Function : rateMovie
180      Purpose  : Update movie rating
181  -----*/
182  void rateMovie() {
183      int id, i;
184      float rating;
185
186      printf("Enter Content ID : ");
187      scanf("%d", &id);
188
189      for(i = 0; i < n; i++) {
190          if(m[i].id==id) {
191              printf("Enter New Rating : ");
192              scanf("%f", &rating);
193
194              m[i].rating = rating;
195
196              printf("Rating Updated Successfully!\n");
197              return;
198          }
199      }
200
201      printf("Content Not Found!\n");
202  }
203

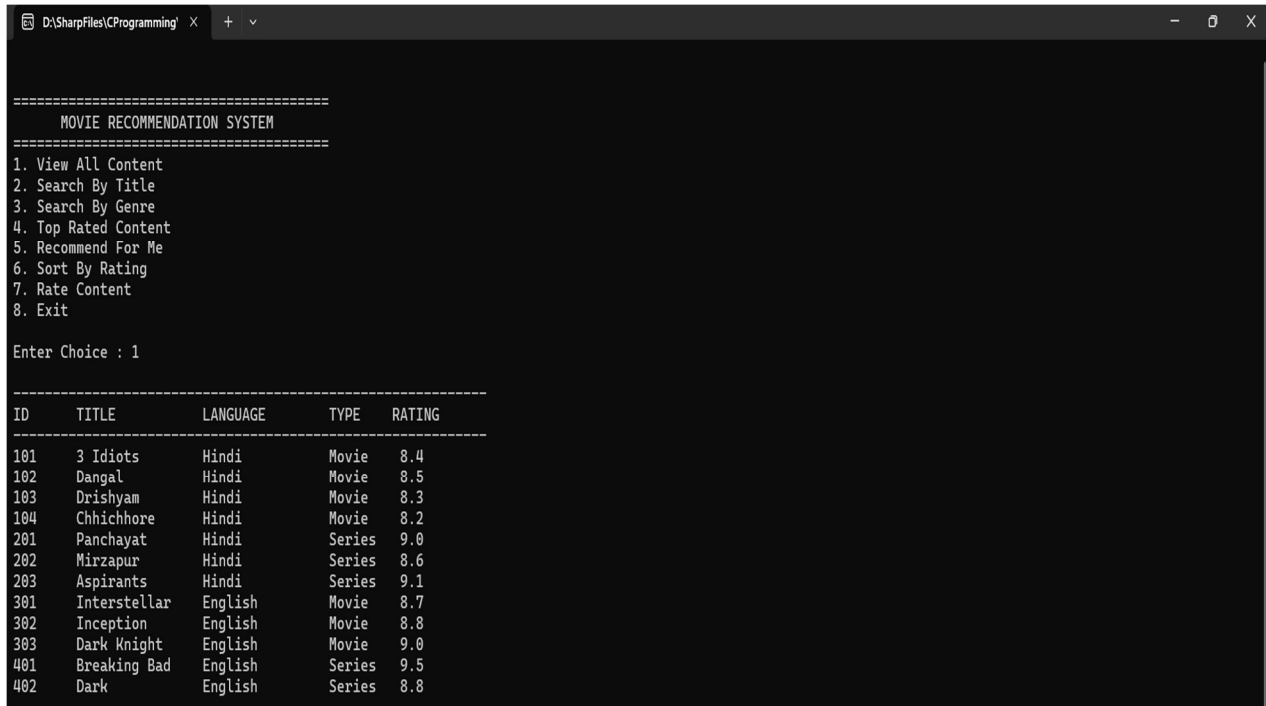
```

```

204  /*-----
205      Main Function
206  -----*/
207  int main() {
208      int choice;
209
210      do {
211          printf("\n\n");
212          printf("=====\\n");
213          printf("      MOVIE RECOMMENDATION SYSTEM\\n");
214          printf("=====\\n");
215
216          printf("1. View All Content\\n");
217          printf("2. Search By Title\\n");
218          printf("3. Search By Genre\\n");
219          printf("4. Top Rated Content\\n");
220          printf("5. Recommend For Me\\n");
221          printf("6. Sort By Rating\\n");
222          printf("7. Rate Content\\n");
223          printf("8. Exit\\n\\n");
224
225          printf("Enter Choice : ");
226          scanf("%d", &choice);
227
228          switch(choice) {
229              case 1:
230                  displayAll();
231                  break;
232              case 2:
233                  searchTitle();
234                  break;
235              case 3:
236                  searchGenre();
237                  break;
238              case 4:
239                  topRated();
240                  break;
241              case 5:
242                  recommendMovie();
243                  break;
244              case 6:
245                  sortRating();
246                  break;
247              case 7:
248                  rateMovie();
249                  break;
250              case 8:
251                  printf("Thank You!\\n");
252                  break;
253              default:
254                  printf("Invalid Choice!\\n");
255          }
256
257      } while(choice != 8);
258
259      return 0;
260  }

```


Output Screenshot:



```
D:\SharpFiles\CProgramming x + v

=====
MOVIE RECOMMENDATION SYSTEM
=====

1. View All Content
2. Search By Title
3. Search By Genre
4. Top Rated Content
5. Recommend For Me
6. Sort By Rating
7. Rate Content
8. Exit

Enter Choice : 1

=====
ID      TITLE      LANGUAGE  TYPE  RATING
=====
101     3 Idiots      Hindi     Movie  8.4
102     Dangal        Hindi     Movie  8.5
103     Drishyam      Hindi     Movie  8.3
104     Chhichhore    Hindi     Movie  8.2
201     Panchayat     Hindi     Series 9.0
202     Mirzapur      Hindi     Series 8.6
203     Aspirants     Hindi     Series 9.1
301     Interstellar  English   Movie  8.7
302     Inception     English   Movie  8.8
303     Dark Knight   English   Movie  9.0
401     Breaking Bad  English   Series 9.5
402     Dark          English   Series 8.8
```

Snake and Ladder in C

Problem Statement:

The objective of this project is to design and implement a **Snake and Ladder** game using the C programming language for two players.

Each player rolls a dice and moves across a 10x10 board (from 1 to 100). The game simulates the movement of players, snakes, and ladders, and determines the winner when a player reaches position **100**. The project helps understand **loops, conditionals, arrays, random number generation**, and **function-based modular programming**.

Concepts Used:

- **Loops:** To iterate through the board and run the game continuously.
- **Arrays:** For the 10x10 conceptual board layout.
- **Functions:** For modularity separate logic for dice roll, board display, and movement.
- **Conditionals (if-else):** To handle player turns, snakes, ladders, and game outcomes.
- **Random Number Generation (rand()):** To simulate dice rolls between 1 and 6.
- **System Functions (system("cls")):** To refresh and display updated game boards clearly.

Short Description of Working:

1. The program starts with both players at position **0**.
2. On each turn, a player presses a key to roll the dice (a random number from 1–6).
3. The player moves forward by that dice value.
4. If the player lands on a **ladder**, they move **up** to a higher position.
5. If the player lands on a **snake**, they slide **down** to a lower position.
6. The board updates dynamically after every move:
 - **'L'** marks ladder positions.
 - **'S'** marks snake positions.
 - **'P1'** and **'P2'** show the current player positions.
7. The first player to reach **exactly 100** wins the game.
8. The game runs in a loop until a winner is declared or the user exits.

snakeAndLadder.c:

```

1  /*-----
2  Project Title   : Snake and Ladder Game
3  Description    : A two-player console-based Snake and Ladder game
4                  that simulates dice rolls, snakes, ladders, and
5                  displays the board dynamically with player positions.
6
7  Rules and Features:
8  - The game begins with any dice value (1-6).
9  - If the dice rolls 6, the player gets another chance.
10 - Snakes and Ladders are shown as 'S' and 'L' on the board.
11 - Players are displayed as 'P1' and 'P2' directly on the board.
12 - Snakes: 99→1, 65→40, 25→9
13   Ladders: 13→42, 60→83, 70→93
14 - The first player to reach exactly 100 wins the game.
15 Concepts Used   : Loops, Functions, Arrays, Conditional Statements,
16                   Switch Case, Random Number Generation.
17 -----*/
18
19
20 #include <stdio.h>
21 #include <stdlib.h>
22
23 /*-----
24 Function: rd
25 Purpose : Generates a random dice value between 1 and 6.
26 -----*/
27 int rd() {
28     int rem;
29     A:
30     rem = rand() % 7;
31     if (rem == 0)
32         goto A;
33     else
34         return rem;
35 }
36
37 /*-----
38 Function: displayChart
39 Purpose : Displays the 10x10 Snake and Ladder board.
40           Marks Snakes (S), Ladders (L), and Player Positions.
41 -----*/
42 void displayChart(int p1, int p2) {
43     int i, j, num, shift = 0;
44
45     printf("\n\t\t\t\t---- SNAKE AND LADDER BOARD ----\n\n");
46
47     // The board contains 10 rows (10x10 grid)
48     for (i = 10; i > 0; i--) {
49         int rowBase = (i - 1) * 10;
50
51         // Left-to-right numbering
52         if (shift % 2 == 0) {
53             for (j = 10; j >= 1; j--) {
54                 num = rowBase + j;
55
56                 // Display player positions
57                 if (num == p1 && num == p2)
58                     printf("P1P2*\t");
59                 else if (num == p1)
60                     printf("P1*\t");
61                 else if (num == p2)
62                     printf("P2*\t");
63
64                 // Display Snakes and Ladders
65                 else if (num == 99 || num == 65 || num == 42 || num == 25)
66                     printf("S\t");
67                 else if (num == 79 || num == 64 || num == 36 || num == 13)
68                     printf("L\t");
69                 else
70                     printf("%d\t", num);
71             }
72         } else { // Right-to-left numbering
73             for (j = 1; j <= 10; j++) {
74                 num = rowBase + j;

```

```

75
76         if (num == p1 && num == p2)
77             printf("P1P2*\t");
78         else if (num == p1)
79             printf("P1*\t");
80         else if (num == p2)
81             printf("P2*\t");
82         else if (num == 99 || num == 65 || num == 42 || num == 25)
83             printf("S\t");
84         else if (num == 79 || num == 64 || num == 36 || num == 13)
85             printf("L\t");
86         else
87             printf("%d\t", num);
88     }
89 }
90
91     shift++;
92     printf("\n\n");
93 }
94
95 printf("-----\n");
96 printf("Player 1 Position: %d\t\t\tPlayer 2 Position: %d\n", p1, p2);
97 printf("-----\n");
98 }
99
100 /*-----
101     Function: movePlayer
102     Purpose : Moves the player according to dice value and applies
103              Snake or Ladder effect if present.
104 -----*/
105 int movePlayer(int curPos, int dice, int playerNum) {
106     curPos += dice;
107
108     // Check if player exceeds 100
109     if (curPos > 100) {
110         curPos -= dice;
111         printf("Player %d exceeded 100! Staying at %d.\n", playerNum, curPos);
112         return curPos;
113     }
114
115     // Snake positions
116     if
117         (curPos == 99) curPos = 1;
118     else if
119         (curPos == 65) curPos = 40;
120     else if
121         (curPos == 25) curPos = 9;
122
123     // Ladder positions
124     else if
125         (curPos == 13) curPos = 42;
126     else if
127         (curPos == 60) curPos = 83;
128     else if
129         (curPos == 70) curPos = 93;
130
131     return curPos;
132 }
133
134 /*-----
135     Function: main
136     Purpose : The main control function to execute the game.
137              Handles player turns, dice rolls, and win conditions.
138 -----*/
139 int main() {
140     int curPos1 = 0, curPos2 = 0, dice;
141     char choice;
142
143     while (1) {
144         system("cls"); // Clears the screen (Windows only)
145         printf("\n\t\t\t\t\t** SNAKE AND LADDER GAME **\n\t\t\t\t\t(Two Player Version)\n");
146         printf("-----\n");

```

```
147
148     displayChart(curPos1, curPos2);
149
150     printf("\n1. Player 1 plays\n2. Player 2 plays\n3. Exit\nEnter your choice: ");
151     scanf(" %c", &choice);
152
153     switch (choice) {
154         case '1':
155             dice = rd();
156             printf("\nPlayer 1 rolled a %d\n", dice);
157             curPos1 = movePlayer(curPos1, dice, 1);
158
159             if (curPos1 == 100) {
160                 displayChart(curPos1, curPos2);
161                 printf("\n Player 1 Wins the Game! \n");
162                 exit(0);
163             }
164             break;
165
166         case '2':
167             dice = rd();
168             printf("\nPlayer 2 rolled a %d\n", dice);
169             curPos2 = movePlayer(curPos2, dice, 2);
170
171             if (curPos2 == 100) {
172                 displayChart(curPos1, curPos2);
173                 printf("\n Player 2 Wins the Game! \n");
174                 exit(0);
175             }
176             break;
177
178         case '3':
179             printf("Thanks for playing!\n");
180             exit(0);
181
182         default:
183             printf("Invalid choice! Try again.\n");
184     }
185
186     printf("\nPress Enter to continue...");
187     getchar();
188     getchar();
189 }
190
191 return 0;
192 }
```

Output Screenshot:

```

** SNAKE AND LADDER GAME **
(Two Player Version)
-----
      ---- SNAKE AND LADDER BOARD ----
100   S    98   97   96   95   94   93   92   91
81    82   83   84   85   86   87   88   89   90
80    79   78   77   76   75   74   73   72   71
61    62   63   64   S    66   67   68   69   L
L     59   58   57   56   55   54   53   52   51
41    42   43   44   45   46   47   48   49   50
40    39   38   37   36   35   34   33   32   31
21    22   23   24   S    26   27   28   29   30
20    P1   18   17   16   15   P2   L    12   11
1     2    3    4    5    6    7    8    9    10
-----
Player 1 Position: 19 |      Player 2 Position: 14
-----
1. Player 1 plays
2. Player 2 plays
3. Exit
Enter your choice:

```